

Step 1 — Writer Drafting Chain — Hints

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried. Each hint has 3 depth levels: **Basic** (the plain idea), **Intermediate** (proper LangChain/LCEL), **Advanced** (what a real production chain adds). Read Basic first even if you already know LangChain — it's the fastest way to spot exactly what each deeper level adds.

- [Hint 1 — The idea, and the exact pieces](#)
- [Hint 2 — The plan, and almost the whole thing](#)

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 1 — The idea, and the exact pieces

Basic Version

This step has no "agent" behavior at all yet — no loop, no tools, no deciding what to do next. It's just: text in (research notes), text out (a draft). That's exactly what an LCEL chain is for: a prompt, piped into a model, piped into something that turns the model's reply into plain text.

Three pieces, connected with `|`: a prompt template, `ChatOpenAI`, and an output parser.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

The README asks for `build_writer_chain() -> Runnable`, so this is a factory function — it builds and returns the chain object, it doesn't run it. That's a deliberate design choice: the caller (Step 1's `main.py`, and later the real Writer agent) decides *when* to invoke it and with *what* input.

The chain itself is LCEL: `prompt | ChatOpenAI(...) | StrOutputParser()`. `ChatPromptTemplate.from_template(...)` takes a template string with a `{notes}` placeholder. The whole chain is invoked with `chain.invoke({"notes": some_notes_string})`. Every LCEL chain implements the `Runnable` interface — that's what `-> Runnable` means, and why `.invoke(...)` always works the same way regardless of what's inside. Keep the model config (`ChatOpenAI(model=..., temperature=...)`) *inside* the factory, not passed in by the caller — simpler is correct at this step. `StrOutputParser()` just extracts `.content` from the model's reply as plain text; without it, `.invoke(...)` returns a full `AIMessage` object instead.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

Think about what happens when this chain is called with `notes=""` or `notes=None` — nothing in the Basic or Intermediate version stops that. The chain will happily invoke the model on empty input and get back a fluent-sounding, content-free draft. That's the same shape of bug the `.env` parsing exercise's required-key check solved: a function that "works" but silently produces something wrong is more dangerous than one that fails loudly, immediately, on bad input — because right now, in Step 1, a blank draft is obvious in your terminal; once this chain is wrapped inside Step 5's full pipeline, a blank draft is just one confusing failure buried three agents deep.

The other real design question: right now the whole instruction — "write a draft, don't invent facts" — lives in one template string handed to `.from_template(...)`. A model tends to follow instructions in a dedicated **system** message more reliably than instructions mixed into one long user-role string. `ChatPromptTemplate.from_messages([("system", ...), ("human", "{notes}")])` splits "how to behave" (system) from "the actual input" (human) — worth doing here specifically because "don't invent facts not in the notes" is a correctness requirement for this project, not just style, and it's about to matter a lot more once Step 3's real Research agent starts feeding this chain genuinely limited information.

The extra pieces:

- A guard clause at the top of `build_writer_chain()`'s caller (or inside a small wrapper), raising `ValueError` if `notes` is empty or whitespace-only — fail before spending an API call on nothing.
- `ChatPromptTemplate.from_messages([("system", WRITER_SYSTEM_PROMPT), ("human", "Research notes:\n{notes}")])` instead of one `from_template(...)` string.
- Keep `temperature` as a named constant near the top of the file, not a magic number buried inline — you'll want to tune it once real output quality matters.

Sketch the guard clause and the system/human split yourself before checking Hint 2.

Difference between Basic, Intermediate, and Advanced: Basic describes the plain idea and lists the tools for the tidy, happy-path case. Intermediate shows the real LCEL syntax and explains what each piece is actually for. Advanced adds the pieces that only matter once this chain has to survive contact with real callers — empty input, and instructions that need to actually stick — which is the difference between a chain that works in a demo and one that would survive being called by Step 5's Supervisor with whatever the Research/Analysis agents happen to hand it.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 2 — The plan, and almost the whole thing

Basic Version

```
make a function build_writer_chain that returns a chain:
    make a prompt template with a {notes} spot in it
    make a chat model
    make a parser that turns the reply into plain text
    connect them with | and return the result

in main.py:
    make 2-3 fake research notes strings
    build the chain
    for each fake notes string:
        call chain.invoke with the notes
        print the draft, read it, check it's reasonable
```

Here's almost the whole thing — just try running it and reading it line by line:

```
from langchain_core.prompts import ChatPromptTemplate
from langchain_openai import ChatOpenAI
from langchain_core.output_parsers import StrOutputParser

def build_writer_chain():
    prompt = ChatPromptTemplate.from_template(
        "Write a short draft based on these research notes:\n{notes}"
    )
    model = ChatOpenAI(model="gpt-4o-mini")
    parser = StrOutputParser()
    return prompt | model | parser
```

Expected output if you run just this (nothing calls the function yet): nothing — defining a function doesn't run it. Add `main.py`'s test calls to actually see a draft print.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

```
agents/writer_chain.py:
def build_writer_chain() -> Runnable:
    prompt = ChatPromptTemplate.from_template(...)
    model = ChatOpenAI(model="gpt-4o-mini", temperature=0.7)
    parser = StrOutputParser()
    return prompt | model | parser

main.py:
chain = build_writer_chain()
fake_notes = [note1, note2, note3]
for notes in fake_notes:
    draft = chain.invoke({"notes": notes})
    print(draft)
```

Test against notes that are deliberately different in shape — short bullet points, a longer paragraph, a notes set with almost nothing in it — so you can see how the chain handles thin input before Research (Step 3) ever exists to feed it real notes. Write the full typed version yourself, with a `WRITER_PROMPT` constant instead of an inline string, then compare against the [Solution](#).

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

Here's almost the hardened version — fill in the missing guard clause yourself:

```
from langchain_core.prompts import ChatPromptTemplate
from langchain_openai import ChatOpenAI
from langchain_core.output_parsers import StrOutputParser
from langchain_core.runnables import Runnable

WRITER_SYSTEM_PROMPT = (
    "You are a content writer. Write a short, clear draft (3-5 paragraphs) "
    "based only on the research notes you are given. Do not invent facts "
    "that aren't in the notes."
)

def build_writer_chain() -> Runnable:
    prompt = ChatPromptTemplate.from_messages([
        ("system", WRITER_SYSTEM_PROMPT),
        ("human", "Research notes:\n{notes}"),
    ])
    model = ChatOpenAI(model="gpt-4o-mini", temperature=0.7)
    return prompt | model | StrOutputParser()

def run_writer_chain(chain: Runnable, notes: str) -> str:
    # your turn: raise ValueError("notes cannot be empty") if notes is
    # empty or whitespace-only, before calling chain.invoke(...)
    ...
    return chain.invoke({"notes": notes})
```

Fill in the guard clause, then compare all 3 of your finished versions against the [Solution](#).

Difference between Basic, Intermediate, and Advanced: same 3 pieces (prompt, model, parser) at 3 completeness levels — Basic just proves the mechanism, Intermediate adds the type contract and a real test loop, Advanced adds a second, separate concern on top: a wrapper function that checks its *input* is actually usable before spending an API call on it, plus a system/human split so "don't invent facts" is a standing instruction, not just one sentence buried in a template string.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Full solution: [Show me the solution](#)